Software Requirements Specification

for

Adaptive Load Balancer

Version 1.0 approved

Prepared by

Thanas R (PES1UG25AM427)
T N Pranav (PES1UG25AM421)
Sushveen Kaur (PES1UG25AM416)

Department of CSE (AI&ML)
PES UNIVERSITY

November 11, 2026

Table of Contents

Revision History **2**
1. Introduction **3**
1.1 Purpose 3
1.2 Document Conventions 3
1.3 Intended Audience and Reading Suggestions 3
1.4 Product Scope 3
1.5 References 4
2. Overall Description **4**
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements **6**
3.1 User Interfaces 6
3.2 Hardware Interfaces 6
3.3 Software Interfaces 7
3.4 Communications Interfaces 7
4. System Features **7**
4.1 Dynamic Layer 4 TCP Load Balancing 7
4.2 Real-Time Socket Telemetry and Health Probing 8
4.3 Automatic Degradation and Server Failure Handling 9
4.4 Server Recovery and Gradual Re-Warming 9
4.5 Performance Benchmarking and Evaluation 10
5. Other Nonfunctional Requirements **11**
5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 11
5.5 Business Rules 12
6. Other Requirements **12**
Appendix A: Glossary **12**
Appendix B: Analysis Models **14**
B.1 Backend Health State Machine 14
B.2 Composite Score Formula 14
B.3 Telemetry JSON Schema 14
Appendix C: Resolved Decisions **15**

Revision History

Name	Date	Reason For Changes	Version
Thanas R, T N Pranav, Sushveen Kaur	November 11, 2026	Initial Version 1.0 Release	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the software requirements for the Adaptive Load Balancer (Version 1.0). The product is an OpenFlow 1.3 software-defined network application integrated with low-level Python TCP socket applications. The scope covers the OpenFlow controller logic, the multi-threaded backend socket workers, telemetry gathering daemons, concurrent client request generators, and the Mininet network topology. This document is intended to serve as a binding reference for development, testing, and evaluation of the system.

1.2 Document Conventions

All requirement statements are tagged with unique identifiers such as REQ-1 and REQ-2. Standard font conventions denote commands, configuration parameters, and API fields. Priorities are indicated as High, Medium, or Low. All metric expressions and algorithms follow standard mathematical notation. The acronym VIP refers to the Virtual IP address exposed to clients. The term controller always refers to the Ryu OpenFlow 1.3 controller unless stated otherwise.

1.3 Intended Audience and Reading Suggestions

This document is prepared for course evaluators, network engineers, system developers, and software testers. Evaluators should focus on Section 2 for architectural scope, Section 4 for detailed functional requirements, and Section 5 for performance evaluation criteria. Developers and testers should examine Section 3 and Section 4 for interface and protocol specifications.

1.4 Product Scope

The Adaptive Load Balancer replaces static, blind distribution with controller-driven, per-flow forwarding decisions. Clients connect to a single Virtual IP address (10.0.0.100:8000). The Ryu SDN controller intercepts each new TCP SYN packet, queries real-time telemetry from all backend servers, computes a composite health score, and installs OpenFlow match-action rules that transparently rewrite the destination IP and MAC to the chosen backend. The system continuously monitors active connections, smoothed latency, and error rates, and it reroutes or rejects flows when a server becomes overloaded or unresponsive. The product demonstrates that programmable network control can outperform static round-robin distribution under variable load conditions.

1.5 References

- OpenFlow Switch Specification, Version 1.3.0, Open Networking Foundation, 2012.
- Ryu SDN Framework Documentation, https://ryu.readthedocs.io
- Mininet: Rapid Prototyping for Software Defined Networks, Lantz et al., ACM HotNets, 2010.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Python Software Foundation, socket module documentation, https://docs.python.org/3/library/socket.html
- Open vSwitch Project, https://www.openvswitch.org

2. Overall Description

2.1 Product Perspective

The Adaptive Load Balancer is a new, self-contained system built to demonstrate SDN-driven traffic management in an academic Mininet environment. It is not a derivative of any existing commercial product. The system consists of four subsystems that interact over standard TCP sockets and OpenFlow 1.3 channels:

- **Client generator:** Python processes running on Mininet host h1, sending concurrent TCP requests to the VIP.
- **Backend servers:** Three multi-threaded Python TCP servers on hosts h2, h3, and h4, each exposing a telemetry port.
- **Ryu SDN controller:** A Python application listening on 127.0.0.1:6653, making per-flow forwarding decisions.
- **Mininet topology:** A single Open vSwitch connecting all hosts, managed by the controller.

2.2 Product Functions

The system provides the following major functions:

- Expose a single Virtual IP (10.0.0.100:8000) as the service endpoint for all clients.
- Intercept TCP SYN packets using OpenFlow packet-in messages and select a backend dynamically.
- Install bidirectional OpenFlow flow rules rewriting IP and MAC headers for the chosen backend.
- Collect load telemetry from each backend over a dedicated TCP socket every 5 seconds.
- Compute a composite score from active connections (weight 0.50), smoothed latency (weight 0.35), and error rate (weight 0.15).

- Mark a backend as UNHEALTHY after 3 consecutive telemetry failures and exclude it from selection.
- Mark a backend as DEGRADED when error rate is at or above 0.25 or latency is at or above 500 ms.
- Reinstate a RECOVERING backend after a 5-second cooldown if telemetry returns successfully.
- Provide a benchmarking harness with light, medium, and heavy load scenarios for comparison against static round-robin.

2.3 User Classes and Characteristics

Network administrators configure and start the system using command-line tools and the Mininet CLI. They are expected to have working knowledge of Python, OpenFlow, and Linux networking. Course evaluators interact with the system by running the provided test scripts and reading printed output or log files. No graphical user interface is required or provided.

2.4 Operating Environment

The system requires the following environment:

- Host OS: Ubuntu 20.04 or later (Linux kernel 5.4+) with Mininet 2.3+ installed.
- Open vSwitch 2.13+ configured as the data-plane switch within Mininet.
- Ryu SDN Framework 4.34+ installed via pip in the host Python environment.
- Python 3.8 or later with the standard socket, threading, and json library modules.
- Network emulation uses a single virtual switch (s1) with four virtual hosts (h1, h2, h3, h4).
- Controller connects to the switch on TCP port 6653 using OpenFlow 1.3.

2.5 Design and Implementation Constraints

The following constraints govern the design and implementation:

- All client-server communication must use Python’s socket module directly with no high-level HTTP or RPC frameworks.
- The controller must use OpenFlow 1.3 message types including PacketIn, FlowMod, and PortStats.
- The topology script must be a standalone Mininet Python file that can be started from the command line.
- All configuration values including VIP, backend IPs, ports, weights, and thresholds must be stored in config/config.json.
- No external databases or message queues may be used; telemetry is exchanged over plain TCP sockets.
- The codebase must pass all unit tests in the tests/ directory using pytest.

2.6 User Documentation

The following documentation is provided alongside the software:

- **README.md:** Step-by-step instructions to install dependencies, start the controller, launch the topology, and run tests.
- **GUIDE.md:** Detailed explanation of each module, the OpenFlow pipeline, the telemetry protocol, and the scoring algorithm.
- **GIST.md:** Concise reference card listing all configuration parameters, port numbers, and load metric definitions.
- **docs/GUIDE.md:** Developer guide covering the directory layout, how to add new backends, and how to extend the controller logic.

2.7 Assumptions and Dependencies

The following assumptions and dependencies apply:

- The host machine runs Linux and supports the creation of Mininet virtual hosts and Open vSwitch bridges.
- Ryu and Mininet are pre-installed before the system is started.
- Backend servers are started inside Mininet before client traffic is generated.
- The controller is started before the Mininet topology is launched so that the switch connects on startup.
- Telemetry port numbers (9002, 9003, 9004) are free on each backend host.
- No external firewall rules block OpenFlow traffic on port 6653.

3. External Interface Requirements

3.1 User Interfaces

The system provides no graphical user interface. All interaction is performed through the terminal. The Mininet CLI allows operators to open xterm windows on individual hosts and run Python scripts. Test output is printed to stdout in structured plain text. Log files are written to the results/ directory.

3.2 Hardware Interfaces

No special hardware is required. The system runs entirely within a Linux virtual machine or a physical machine with at least 2 GB of RAM and 2 CPU cores. All network interfaces are emulated by Open vSwitch and the Mininet virtual host namespace mechanism. No physical network adapters are accessed.

3.3 Software Interfaces

The system interfaces with the following external software components:

- **Mininet 2.3+:** Provides the virtual topology, host namespaces, and the Mininet Python API.
- **Open vSwitch 2.13+:** Acts as the OpenFlow-capable data-plane switch between hosts.
- **Ryu 4.34+:** Provides the OpenFlow 1.3 application framework; the controller imports ryu.base, ryu.controller, ryu.ofproto, and ryu.lib.packet.
- **Python 3.8+ standard library:** socket, threading, json, time, logging, collections, and argparse.
- **pytest 7+:** Used to discover and execute all unit tests in the tests/ directory.

3.4 Communications Interfaces

The following network communication interfaces are defined:

Interface	Protocol	Port	Direction	Purpose
Client to VIP	TCP	8000	Client to Switch	Application requests
Controller to Switch	TCP	6653	Bidirectional	OpenFlow 1.3 control
Controller to h2	TCP	9002	Controller to h2	Telemetry polling
Controller to h3	TCP	9003	Controller to h3	Telemetry polling
Controller to h4	TCP	9004	Controller to h4	Telemetry polling
Backend to Client	TCP	Ephemeral	h2/h3/h4 to h1	Response data

4. System Features

4.1 Dynamic Layer 4 TCP Load Balancing

4.1.1 Description and Priority

Priority: High. The core feature of the system is the ability to intercept every new TCP client flow at the OpenFlow switch and forward it to the least-loaded backend server without the client knowing the real server address. The VIP (10.0.0.100:8000) is the only address exposed to clients.

4.1.2 Stimulus/Response Sequences

1. The client sends a TCP SYN to 10.0.0.100:8000.
2. The switch has no matching flow rule and sends a PacketIn message to the controller.
3. The controller queries the current telemetry snapshot for all healthy backends.
4. The controller computes the composite score and selects the backend with the lowest score.

5. The controller installs two FlowMod rules: one rewriting destination IP and MAC to the chosen backend for client-to-server traffic, and one rewriting source IP and MAC back to the VIP for server-to-client traffic.
6. The controller sends a PacketOut to forward the buffered SYN to the chosen backend.
7. Subsequent packets in the same flow match the installed rules and are forwarded in the data plane without controller involvement.

4.1.3 Functional Requirements

REQ-1: The controller MUST expose a virtual service at IP 10.0.0.100 and TCP port 8000.

REQ-2: The controller MUST select a backend using a composite score computed from active connections, smoothed latency, and error rate with weights 0.50, 0.35, and 0.15 respectively.

REQ-3: The controller MUST install bidirectional flow rules with an idle timeout of 30 seconds.

REQ-4: The controller MUST NOT select a backend whose health state is UNHEALTHY.

REQ-5: The system MUST handle at least 50 concurrent client connections without dropping packets due to controller overload.

4.2 Real-Time Socket Telemetry and Health Probing

4.2.1 Description and Priority

Priority: High. Each backend server exposes a telemetry endpoint on a dedicated TCP port (9002, 9003, 9004). The controller polls each endpoint every 5 seconds and parses a JSON object containing the server's current active connection count, exponential moving average latency, and error rate.

4.2.2 Stimulus/Response Sequences

1. The controller's telemetry thread opens a TCP connection to backend_ip:telemetry_port.
2. The backend responds immediately with a JSON object: {"active": N, "latency_ms": F, "errors": F}.
3. The controller parses the JSON, updates its in-memory snapshot, and closes the connection.
4. If the connection is refused or the JSON is malformed, the controller increments the backend's failure counter.

4.2.3 Functional Requirements

REQ-6: Each backend MUST expose a TCP telemetry listener on its designated port at all times while running.

REQ-7: Telemetry responses MUST be valid JSON with keys: active (integer), latency_ms (float), errors (float in [0.0, 1.0]).

REQ-8: The controller MUST poll telemetry every 5 seconds per backend.

REQ-9: Latency values MUST be smoothed using an exponential moving average with alpha equal to 0.25.

REQ-10: The controller MUST update the composite score snapshot before every new flow assignment.

4.3 Automatic Degradation and Server Failure Handling

4.3.1 Description and Priority

Priority: High. The controller must detect when a backend is overloaded (DEGRADED) or completely unreachable (UNHEALTHY) and adjust its forwarding decisions accordingly. DEGRADED backends remain eligible for new flows but are penalized in the scoring formula. UNHEALTHY backends are excluded entirely.

4.3.2 Stimulus/Response Sequences

1. The controller receives telemetry with error_rate >= 0.25 or latency_ms >= 500: backend moves to DEGRADED.
2. The controller fails to receive valid telemetry 3 consecutive times: backend moves to UNHEALTHY.
3. The controller routes all subsequent new flows away from UNHEALTHY backends.
4. If only one backend remains healthy, the controller routes all flows to it.
5. If all backends are UNHEALTHY, the controller sends a TCP RST via PacketOut to reject the connection.

4.3.3 Functional Requirements

REQ-11: A backend MUST be marked UNHEALTHY after 3 consecutive telemetry failures.

REQ-12: A backend MUST be marked DEGRADED when error_rate >= 0.25 or latency_ms >= 500.

REQ-13: UNHEALTHY backends MUST NOT receive new flows until they recover.

REQ-14: The controller MUST log every state transition with a timestamp and the backend identifier.

REQ-15: If all backends are UNHEALTHY, the controller MUST reject new client connections.

4.4 Server Recovery and Gradual Re-Warming

4.4.1 Description and Priority

Priority: Medium. After a backend recovers from an UNHEALTHY state, it must not immediately receive a full share of traffic. A 5-second cooldown (RECOVERING state) applies during which the backend receives reduced traffic until its metrics stabilize.

4.4.2 Stimulus/Response Sequences

1. A previously UNHEALTHY backend begins responding to telemetry polls successfully.
2. The controller moves it to RECOVERING and starts a 5-second cooldown timer.
3. During RECOVERING, the backend's composite score is artificially inflated to reduce its selection probability.
4. After 5 seconds with continued healthy telemetry, the backend moves to HEALTHY.
5. The controller resumes normal score-based selection including the recovered backend.

4.4.3 Functional Requirements

REQ-16: A backend MUST enter RECOVERING state upon first successful telemetry response after UNHEALTHY.

REQ-17: The RECOVERING state MUST last at least 5 seconds.

REQ-18: The controller MUST apply a score penalty during the RECOVERING state to reduce flow assignment probability.

REQ-19: The backend MUST transition to HEALTHY only after RECOVERING completes with no further failures.

4.5 Performance Benchmarking and Evaluation

4.5.1 Description and Priority

Priority: High. The system must include a structured benchmarking harness that runs three load conditions (light, medium, heavy) and compares the adaptive controller against a static round-robin baseline. Results must include per-server request counts, mean response time, and error rate under each condition.

4.5.2 Stimulus/Response Sequences

1. The operator selects a scenario script (scenario1 through scenario6).
2. The harness launches N concurrent client threads, each sending M sequential requests.
3. After all clients complete, the harness prints per-server request counts, mean latency, standard deviation, and error count.
4. The operator runs the same scenario with the static round-robin topology for baseline comparison.
5. Results are stored in the results/ directory.

4.5.3 Functional Requirements

REQ-20: The benchmarking harness MUST support at least three load conditions: light (5 clients), medium (20 clients), and heavy (50 clients).

REQ-21: Each scenario MUST report per-server request count, mean response time in milliseconds, standard deviation, and total error count.

REQ-22: The harness MUST include a static round-robin baseline scenario for fair comparison.

REQ-23: All scenario results MUST be reproducible: running the same scenario twice under the same conditions MUST produce results within 10 percent of each other.

REQ-24: Graphs or tables summarizing the comparison MUST be stored in the results/ directory.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system MUST handle 50 concurrent TCP client connections without packet loss caused by controller processing delays.
- Controller telemetry polling MUST complete within 1 second per backend per polling cycle.
- Flow rule installation MUST complete within 100 milliseconds of the PacketIn event.
- Mean response time under light load (5 clients) MUST be below 300 milliseconds.
- The system MUST demonstrate measurably lower error rates under adaptive balancing compared to static round-robin under heavy load.

5.2 Safety Requirements

This system is an academic prototype and does not control physical infrastructure. No safety-critical requirements apply. The system must not crash the host operating system or leave orphaned Mininet processes running after shutdown. All processes must terminate cleanly when the Mininet cleanup command is issued.

5.3 Security Requirements

- The telemetry port on each backend MUST only accept connections from 127.0.0.1 (loopback) in the Mininet namespace to prevent external access.
- The OpenFlow control channel MUST run on the loopback interface (127.0.0.1:6653) and MUST NOT be exposed externally.
- No authentication credentials or personal data are stored or transmitted by this system.
- The system MUST NOT execute arbitrary shell commands received from network peers.

5.4 Software Quality Attributes

- **Reliability:** The controller must continue operating when one or two backends fail, routing remaining traffic to healthy servers.

- **Maintainability:** Configuration values must be isolated in config/config.json. Adding a new backend requires only a configuration change and no code modifications.
- **Testability:** Every core module must have corresponding unit tests in tests/. All tests must pass with pytest.
- **Reproducibility:** A person who clones the repository and follows the README must be able to reproduce all test results.
- **Readability:** Every Python module must include a module-level docstring and inline comments explaining non-obvious logic.

5.5 Business Rules

- The adaptive balancer MUST demonstrate better load distribution than static round-robin under at least one load condition in the benchmark.
- The system MUST adhere to the team assignment rules: TCP sockets, Python socket module, Mininet, Open vSwitch, OpenFlow 1.3, and one SDN controller (Ryu).
- Every team member must own and be able to explain one visible module: application, controller, or topology and testing.

6. Other Requirements

The following requirements do not fall neatly into the categories above but are mandatory for project acceptance:

- The GitHub repository MUST include the topology/, controller/, app/, config/, tests/, results/, and docs/ directories as specified in the assignment.
- The README.md MUST include exact run steps that allow a reviewer to reproduce the system from a fresh clone with no additional instructions.
- The viva demonstration MUST show the controller dynamically changing forwarding rules while clients are actively sending requests.
- All submitted files MUST be free of placeholder text, commented-out dead code, and debugging print statements that were not intentionally included.
- The final submission MUST include at least one performance graph or summary table in the results/ directory.

Appendix A: Glossary

VIP (Virtual IP): The single IP address (10.0.0.100) exposed to all clients. The real backend IPs are hidden behind it.

SDN: Software-Defined Networking. An architecture that separates the control plane (controller) from the data plane (switch).

OpenFlow: A standardized protocol between an SDN controller and a network switch. Version 1.3 is used in this project.

FlowMod: An OpenFlow message sent by the controller to install, modify, or delete a flow rule on the switch.

PacketIn: An OpenFlow message sent by the switch to the controller when a packet arrives with no matching flow rule.

PacketOut: An OpenFlow message sent by the controller to instruct the switch to forward a specific packet.

EMA: Exponential Moving Average. A smoothing technique applied to latency values with alpha equal to 0.25.

Composite Score: A weighted sum of active connections (0.50), smoothed latency (0.35), and error rate (0.15) used to rank backends.

HEALTHY: A backend state indicating normal operation and eligibility for new flows.

DEGRADED: A backend state indicating elevated error rate or latency. The backend remains eligible but is penalized in scoring.

UNHEALTHY: A backend state indicating 3 consecutive telemetry failures. The backend is excluded from new flow assignments.

RECOVERING: A backend state between UNHEALTHY and HEALTHY. The backend is re-warming for 5 seconds before returning to full eligibility.

Telemetry: Runtime load metrics (active connections, latency, error rate) reported by each backend server over a dedicated TCP port.

Mininet: A network emulation tool that creates virtual hosts, switches, and links on a single Linux machine.

Open vSwitch (OVS): A software-based OpenFlow-capable Ethernet switch used as the data plane in Mininet.

Ryu: An open-source Python SDN controller framework supporting OpenFlow 1.3.

Round-Robin: A static load balancing strategy that distributes requests to backends in a fixed cyclic order with no regard for load.

Appendix B: Analysis Models

B.1 Backend Health State Machine

The controller maintains one of four health states for each backend:

State	Entry Condition	Exit Condition	Effect on Routing
HEALTHY	Initial state or after RECOVERING completes	Error rate >= 0.25 or latency >= 500 ms	Fully eligible for new flows
DEGRADED	Error rate >= 0.25 or latency >= 500 ms	Metrics return below thresholds	Eligible but score-penalized
UNHEALTHY	3 consecutive telemetry failures	Telemetry succeeds once	Excluded from all new flows
RECOVERING	First success after UNHEALTHY	5 seconds elapse with no failure	Score-penalized; limited eligibility

B.2 Composite Score Formula

Given a backend with active connection count A, smoothed latency L (in milliseconds), and error rate E (in [0.0, 1.0]), the composite score S is computed as:

```
S = 0.50 * normalize(A) + 0.35 * normalize(L) + 0.15 * E
```

where normalize(x) scales x linearly between 0 and 1 relative to the observed range across all healthy backends. The backend with the lowest S is selected for the next flow.

B.3 Telemetry JSON Schema

```
{
  "active": <integer>,       // current active TCP connections
  "latency_ms": <float>,     // EMA-smoothed response time in ms
  "errors": <float>          // error rate in [0.0, 1.0]
}
```

Appendix C: Resolved Decisions

The items that were open in earlier drafts are now closed. The decisions are listed below.

ID	Topic	Decision
D-1	Benchmark graph format and location	Graphs are saved as PNG files created with matplotlib. The same data is also saved as a plain text summary table. Both are stored in the results/ directory.
D-2	Number of backends	The controller supports any number of backends. Backend IPs and telemetry ports are read from config/config.json, so adding a backend needs no code change. The default setup uses three backends.
D-3	Minimum polling interval	The polling interval is set in config/config.json. The default is 5 seconds and the minimum allowed value is 1 second, which matches the 1 second polling limit in Section 5.1.
D-4	Hard timeouts for flow rules	Flow rules use an idle timeout of 30 seconds only. A hard timeout is not used because it would remove the rules of active long-lived connections and break them.